

Bài thực hành số 4: Tạo và triển khai Token ERC-20 trên Ethereum

1. Mục tiêu

Bài thực hành nhằm giúp sinh viên nắm vững các kiến thức và kỹ năng cơ bản trong việc tạo một token trên nền tảng Ethereum bằng ngôn ngữ Vyper. Sau khi hoàn thành, sinh viên có thể:

- Biết cách tạo một token smart contract.
- Hiểu và triển khai token theo chuẩn ERC-20.
- Bán Token.

2. Thời gian làm bài

Dự kiến: 3 giờ.

3. Cơ sở lý thuyết và bài tập tại lớp

A. Tạo một token smart contract đơn giản

Token chính là ứng dụng đột phá thực sự trên Ethereum. Token có thể đại diện cho nhiều thứ, từ tiền ảo, điểm tích lũy, cổ phần trong công ty cho đến đại diện cho các tài sản trong thế giới thực. Ứng dụng nổi tiếng nhất của token là stablecoin - đại diện cho tiền pháp định, thường là Đô la Mỹ. Ví dụ như USDC và USDT, trong đó 1 USDC đại diện cho 1 Đô la Mỹ. Người dùng có thể đổi 1 USDC lấy 1 Đô la Mỹ.

Sử dụng framework APE để tạo project mới đặt tên là HelloToken. Chụp hình cấu trúc project bỏ vào file báo cáo để nộp.

Tạo 1 file smart contract trong thư mục contracts có tên: VerySimpleToken.vy có nội dung đoạn code như sau:

```
1  # @version ^0.4.3
2
3  balances: public(HashMap[address, uint256])
4
5  @deploy
6  def __init__():
7      self.balances[msg.sender] = 10000
8
9  @external
10 def transfer(_to: address, _amount: uint256) -> bool:
11     assert self.balances[msg.sender] >= _amount
12     self.balances[msg.sender] -= _amount
13     self.balances[_to] += _amount
14     return True
```

Hình 1. Đoạn chương trình smart contract VerySimpleContract.vy

Sau khi viết xong đoạn code trong hình 1, các em giải thích đoạn code trên:

1. Cho biết người triển khai nhận được bao nhiêu token? Các địa chỉ khác nhận được bao nhiêu token?
2. Hàm được sử dụng để trao đổi token giữa các đối tượng là gì? Giải thích ý nghĩa các tham số.
3. Tổng số token tối đa được tạo ra và thực hiện trao đổi trong đoạn chương trình số 1 là bao nhiêu?

Sau khi thực hiện tìm hiểu đoạn chương trình, bây giờ các em sẽ viết các chương trình kiểm thử. Nhưng trước đó các em phải biên dịch smart contract này bằng câu lệnh: `ape compile`.

Để kiểm tra hoạt động của đoạn chương trình trên các em được yêu cầu tạo ra 2 file python sau: `conftest.py` & `test_VerySimpleToken.py` trong thư mục `tests`.

```
1  import pytest
2
3  @pytest.fixture
4  def deployer(accounts):
5      return accounts[0]
6
7  @pytest.fixture
8  def contract(deployer, project):
9      return deployer.deploy(project.VerySimpleToken)
```

Hình 2. Nội dung của file `conftest.py`

```
1  def test_balances(contract, deployer):
2      balance = contract.balances(deployer)
3      assert balance == 10000
4
5  def test_transfer(contract, deployer, accounts):
6      destination_account = accounts[1]
7      balance = contract.balances(destination_account)
8      assert balance == 0
9      balance = contract.balances(deployer)
10     assert balance == 10000
11     contract.transfer(destination_account, 200, sender=deployer)
12     balance = contract.balances(destination_account)
13     assert balance == 200
14     balance = contract.balances(deployer)
15     assert balance == 9800
```

Hình 3. Nội dung của file `test_VerySimpleToken.py`

Các em tìm hiểu chức năng của các hàm trong 2 file python `conftest.py` & `test_VerySimpleToken.py`. Sau đó ghi lại chức năng của các hàm này trong file báo cáo nộp lại:

1. Phương thức `test_balances` để làm gì?

2. Phương thức `test_transfer` để kiểm tra hành động gì? Có thực hiện thao tác gì trước khi hành động đó và sau hành động đó hay không? Nếu có các em ghi chi tiết hành động và ý nghĩa đó là gì?

Sau khi tìm hiểu xong các em thực hiện chạy chương trình bằng đoạn lệnh sau:
`py.test tests/test_VerySimpleToken.py`

Các em chụp kết quả chạy đoạn lệnh chương trình trên đưa vào file báo cáo.

Các em thực hiện so sánh việc tạo ra token dựa trên smart contract và cách tạo ra token dựa trên web truyền thống? Nêu ý tưởng, điểm mạnh, điểm yếu của từng phương pháp.

Ứng dụng token là loại ứng dụng được sử dụng phổ biến trong công nghệ blockchain. Blockchain sở hữu các đặc tính về tính minh bạch và khả năng chống kiểm duyệt, rất hữu ích cho các ứng dụng token liên quan đến các giá trị. Ở đây, "giá trị" được đề cập trong nội dung bài này là những thứ như tiền tệ. Ví dụ về các ứng dụng giao dịch giá trị bao gồm các ứng dụng ngân hàng cần theo dõi số dư của người dùng.

Việc đảm bảo các giá trị này không thể bị sai lệch là vô cùng quan trọng. Chẳng hạn, nếu chỉ nói đến giá trị của các tài sản trong một trò chơi điện tử bị lỗi, điều đó có thể chấp nhận được và không gây ra vấn đề lớn. Tuy nhiên, khi nói đến các giá trị tiền tệ, mọi người sẽ không xem nhẹ nếu những giá trị đó bị đánh cắp hoặc thao túng.

Điểm đặc biệt của hợp đồng thông minh là tính linh hoạt rất cao. Các em có thể tạo ra bất kỳ quy tắc nào mình muốn. Trong ví dụ về hợp đồng thông minh `VerySimpleToken`, số lượng token được cố định bằng một con số cứng trong phương thức khởi tạo. Tuy nhiên, không có gì ngăn cản bạn bổ sung các quy tắc khác, như khả năng phát hành thêm token mới vào lượng token hiện có. Tính năng này có thể sẽ hữu ích để mô phỏng lạm phát.

Bây giờ, các em tạo một tệp mới có tên `TokenWithInflation.vy` trong thư mục `contracts`. Sau đó, thêm đoạn mã như hình 4 vào.

Các em có nhiệm vụ tìm hiểu đoạn chương trình này và trả lời các câu hỏi sau vào file báo cáo.

1. Biến `owner` có kiểu dữ liệu là gì? Trong phương thức khởi tạo `init`, biến này được thiết lập địa chỉ cho ai?
2. Phương thức `mint`, ai có thể gọi được phương thức này? Phương thức `mint` chấp nhận một đối số `new_supply` (nguồn cung mới), ý nghĩa của đối số này là gì và được sử dụng để điều khiển gì?

Ví dụ `TokenWithInflation.vy` cho thấy khả năng tăng token cũng có thể được nhìn nhận tích cực. Đây được gọi là lạm phát. Nhiều quốc gia có thể in tiền. Nếu hành động này không được thực hiện quá mức, người dân vẫn có thể chấp nhận nó. Mọi người có thể tham gia vào hợp đồng thông minh này và kiểm tra xem chính phủ có phát hành token quá mức hay không. Nếu như người dân không hài lòng với hành động của chính phủ, họ luôn có thể chọn không tham gia vào hợp đồng thông minh của chính phủ.

```

# @version ^0.4.3

balances: public(HashMap[address, uint256])
owner: public(address)
@deploy
def __init__():
    self.balances[msg.sender] = 10000
    self.owner = msg.sender

@external
def transfer(_to: address, _amount: uint256) -> bool:
    assert self.balances[msg.sender] >= _amount
    self.balances[msg.sender] -= _amount
    self.balances[_to] += _amount
    return True

@external
def mint(_new_supply: uint256):
    assert msg.sender == self.owner
    self.balances[msg.sender] = _new_supply

```

Hình 4. Đoạn chương trình minh họa kiểm soát lạm phát TokenWithInflation

B. Token theo chuẩn ERC-20

Về mặt kỹ thuật, ERC-20 là một tiêu chuẩn để tạo token trên Ethereum. Token đơn giản được tạo trong mục A của bài thực hành này không đáp ứng tiêu chuẩn ERC-20. Đúng vậy, token tạo ra từ mã file VerySimpleToken là một hợp đồng thông minh token kỹ thuật số, nhưng không phải là token ERC-20. Tiêu chuẩn ERC-20 là một trong nhiều lý do giải thích cho sự gia tăng nhanh chóng của các loại tiền mã hóa mới. Tuy nhiên, ERC-20 không phải là yêu cầu bắt buộc để tạo token trên Ethereum.

Tuy nhiên, một số lợi ích nếu bạn tuân theo tiêu chuẩn token ERC-20:

- Thứ nhất, giúp người dùng dễ dàng kiểm tra hợp đồng thông minh của bạn.
- Thứ hai, token ERC-20 của bạn sẽ được nhận dạng tự động bởi ví Ethereum như MetaMask.
- Thứ ba, các sàn giao dịch tiền mã hóa sẽ dễ dàng niêm yết token của bạn hơn. Về cơ bản, nó giúp cuộc sống của mọi người dễ dàng hơn.
- Thứ tư, tiêu chuẩn này cho phép các hợp đồng thông minh sàn giao dịch phi tập trung có thể mua bán token ERC-20.

Để tạo một hợp đồng thông minh token ERC-20, có 2 cách một là em sử dụng interface ERC-20; hai là các em phải tự định nghĩa interface ERC-20 để sử dụng.

1. Khi tạo interface ERC-20 theo chuẩn thì các em phải đảm bảo đủ các phương thức như hình số 5. Bên cạnh đó có các phương thức tùy chọn trong quá trình xây dựng ERC-20 như hình số 6.
2. Các em có thể implement và sử dụng interface ERC-20 được định nghĩa sẵn như hình số 7.

```

# @version ^0.4.3
# ERC-20 Token Standard Interface
# https://eips.ethereum.org/EIPS/eip-20
@external
@view
def totalSupply() -> uint256:
    pass
@external
@view
def balanceOf(_owner: address) -> uint256:
    pass
@external
def transfer(_to: address, _value: uint256) -> bool:
    pass
@external
def transferFrom(_from: address, _to: address, _value: uint256) -> bool:
    pass
@external
def approve(_spender: address, _value: uint256) -> bool:
    pass
@external
@view
def allowance(_owner: address, _spender: address) -> uint256:
    pass

```

Hình 5. Interface ERC-20 tự định nghĩa.

```

@external
@view
def name() -> String[32]:
    pass
@external
@view
def symbol() -> String[16]:
    pass
@external
@view
def decimals() -> uint8:
    pass

```

Hình 6. Các phương thức tùy chọn trong quá trình tạo Token theo chuẩn ERC-20.

Các em có thể chọn một trong 2 cách trên để hiện thực smart contract theo chuẩn ERC-20. Sau khi hiện thực thì nhiệm vụ của các em tìm hiểu ý nghĩa, vai trò, cách dùng của các hàm trong smart contract theo chuẩn ERC-20 và viết báo cáo?

Bên cạnh tiêu chuẩn ERC-20 và ERC20Detailed được sử dụng trong bài thực hành, còn có một số tiêu chuẩn khác như: ERC20Mintable, ERC20Burnable, ERC20Capped, ERC20Pausable? Các em tìm hiểu về mặt kỹ thuật, cách dùng và áp dụng trong thực tế như thế nào để viết báo cáo.

```
# @version ^0.4.3
from ethereum.ercs import IERC20
from ethereum.ercs import IERC20Detailed

implements: IERC20
implements: IERC20Detailed

event Transfer:
    sender: indexed(address)
    receiver: indexed(address)
    value: uint256

event Approval:
    owner: indexed(address)
    spender: indexed(address)
    value: uint256

name: public(String[32])
symbol: public(String[32])
decimals: public(uint8)

balanceOf: public(HashMap[address, uint256])
allowance: public(HashMap[address, HashMap[address, uint256]])
totalSupply: public(uint256)
```

a. Khai báo interface

```
@deploy
def __init__(_name: String[32], _symbol: String[32], _decimals: uint8, _supply: uint256):
    init_supply: uint256 = _supply * 10 ** convert(_decimals, uint256)
    self.name = _name
    self.symbol = _symbol
    self.decimals = _decimals
    self.balanceOf[msg.sender] = init_supply
    self.totalSupply = init_supply
    log Transfer(empty(address), msg.sender, init_supply)

@external
def transfer(_to: address, _value: uint256) -> bool:
    self.balanceOf[msg.sender] -= _value
    self.balanceOf[_to] += _value
    log Transfer(msg.sender, _to, _value)
    return True

@external
def transferFrom(_from: address, _to: address, _value: uint256) -> bool:
    self.balanceOf[_from] -= _value
    self.balanceOf[_to] += _value
    self.allowance[_from][msg.sender] -= _value
    log Transfer(_from, _to, _value)
    return True

@external
def approve(_spender: address, _value: uint256) -> bool:
    self.allowance[msg.sender][_spender] = _value
    log Approval(msg.sender, _spender, _value)
    return True
```

b. Các hàm bắt buộc có trong smart contract theo chuẩn ERC-20

Hình 7. Cách sử dụng ERC-20 từ thư viện có sẵn.

C. Bán tokens

Trong nội dung này, các em tạo một hợp đồng thông minh theo chuẩn ERC-20 và tạo một phương thức cho phép mọi người mua token bằng ETH. Ví dụ: các em muốn bán token của mình để đổi lấy ETH. Việc tạo token gây quỹ cộng đồng trên nền tảng Ethereum dễ dàng hơn nhiều so với nền tảng Bitcoin. Các em đã biết cách tạo một phương thức trong hợp đồng thông minh để nhận ETH. Các em cũng biết cách tăng số dư token của một số tài khoản. Để bán token, các em chỉ cần kết hợp hai điều đó. Đây là cốt lõi của quy trình phát hành tiền mã hóa lần đầu (ICO). Đồng tiền của Ethereum có giá trị thực. Mặc dù giá của ether biến động, nhưng tại thời điểm các em làm bài thực hành này thì 1 ETH là bao nhiêu? các em viết vào báo cáo con số này.

Bây giờ, các em có thể suy nghĩ mọi người sẽ trả tiền thật để mua ETH chứ không phải token do các em tạo ra. Để khiến token của các em có giá trị, trước tiên các em phải làm cho nó hữu ích, đại diện giá trị có một loại tài sản nào đó. Tuy nhiên, để làm được điều đó, các em cần vốn. Vậy tại sao không bán một phần token của các em (ví dụ: 60%) cho những người chấp nhận sớm? Họ có thể mua token do các em tạo ra bằng ETH. Sau đó, các em có thể rút ETH để thuê thêm lập trình viên và thuê văn phòng để phát triển loại tiền mã hóa mới của mình. Đây là ý tưởng cơ bản.

Bây giờ, các em sẽ thử xây dựng hợp đồng thông minh bán token đại chúng (crowd sale); nó giống với mã nguồn hợp đồng thông minh ERC-20 trong phần B mà các em đã hiện thực. Các em đặt tên hợp đồng thông minh này là CrowdSaleToken_MSSV.vy và lưu nó trong thư mục contracts.

Các sự kiện Transfer và Approval là các sự kiện theo tiêu chuẩn ERC-20. Sự kiện Payment là sự kiện tùy chỉnh sẽ được ghi nhận khi một người mua mua token từ hợp đồng thông minh này.

Biến trạng thái ethBalances là một biến ánh xạ theo dõi số lượng ETH mà người mua đã lưu trữ trong hợp đồng thông minh. Biến trạng thái beneficiary được sử dụng để lưu trữ thông tin chủ sở hữu hợp đồng thông minh sẽ nhận ETH từ người mua; nói cách khác, đây là người bán. Biến trạng thái minFundingGoal lưu thông tin mức ETH tối thiểu muốn nhận cho hợp đồng thông minh này. Biến trạng thái maxFundingGoal lưu thông tin mức ETH tối đa muốn nhận từ việc bán token. Điều này sẽ giúp ngăn người mua mua token khi đã đạt được mục tiêu này. Biến trạng thái amountRaised sử dụng để lưu số lượng ETH đã nhận được từ việc bán token. Biến trạng thái deadline được sử dụng thiết lập ngày muộn nhất người mua có thể mua token. Biến trạng thái price sử dụng thiết lập giá của token tính bằng ETH. Biến trạng thái fundingGoalReached cho biết mục tiêu gây quỹ tối thiểu có đạt được hay không. Biến trạng thái crowdsaleClosed cho biết việc bán token đã bị dừng hay chưa.

Đầu tiên, các em thiết lập các thuộc tính tiêu chuẩn của hợp đồng thông minh ERC-20. Tổng nguồn cung token trong hợp đồng thông minh này được xác định bởi đối số, như trong hợp đồng thông minh ERC-20 trong phần B. Các em thiết lập việc gây quỹ ít nhất 30 ether và tối đa 50 ether.

Thời hạn được đặt là 100 ngày kể từ thời điểm hợp đồng thông minh được triển khai trên blockchain. Thuộc tính block.timestamp cho phép lấy thông tin thời gian hiện tại hoặc thời gian khối chứa mã hợp đồng thông minh này được xác nhận.

Giá của một token được đặt là 0.01 ether. Điều này có nghĩa là 1 ether có thể mua 100 token của bạn.

Đây là phương thức mà người dùng có thể sử dụng để mua token. __default__ là hàm fallback mặc định. Nếu ai đó không thực thi một phương thức trên hợp đồng thông minh và thanh toán ether, hàm này sẽ được thực thi. Thực tế, các em không nhất thiết phải sử dụng hàm mặc định để chấp nhận thanh toán; mà các em có thể sử dụng một phương thức thông thường, giống như các em đã làm trong các hợp đồng thông minh ở mục B.

Trong phương thức thanh toán này, các em đảm bảo rằng người mua không phải là beneficiary, rằng đợt bán đại chúng vẫn đang diễn ra và rằng số tiền đã gây được cùng với ether được gửi đến phương thức này không vượt quá mục tiêu gây quỹ tối đa là 50 ether. Cuối cùng, mỗi hành động mua phải có giá ít nhất 0.01 ether. Sau đó, các em tăng số dư ether cho người mua này đồng thời tăng số lượng ether đã gây được. Sau đó các em kiểm tra số lượng token họ đã mua bằng cách chia số lượng ether cho giá của một token.

Cuối cùng, các em phải tăng số dư token cho người mua này và giảm số dư token cho chủ sở hữu hợp đồng thông minh.

Trong quá trình bán token, các em luôn phải đảm bảo rằng phương thức checkGoalReached chỉ có thể được thực thi thành công nếu thời hạn đã hết. Nếu số tiền

gây được nhiều hơn mục tiêu gây quỹ tối thiểu, các em đặt biến `fundingGoalReached` thành `True`. Sau đó, các em đặt biến `crowdsaleClosed` thành `True`.

Để đơn giản, các em chỉ kiểm tra xem biến `block.timestamp` có lớn hơn thời hạn hay không. Tuy nhiên, `timestamp` trong khối có thể được điền bất cứ thứ gì mà thợ đào muốn; nó không nhất thiết phải là thời gian hiện tại khi khối được xác nhận. Tuy nhiên, nếu thợ đào đưa ra `timestamp` trong quá khứ làm giá trị cho `block.timestamp`, tất cả các thợ đào khác sẽ từ chối nó. Tương tự, nếu thợ đào đưa ra một `timestamp` trong tương lai rất xa (ví dụ: trước một năm) làm giá trị cho `block.timestamp`, tất cả các thợ đào khác cũng sẽ từ chối nó. Để làm cho quá trình kiểm tra thời hạn an toàn hơn, các em phải kết hợp nó với `block.number` để kiểm tra xem có bao nhiêu khối đã được xác nhận kể từ khi hợp đồng thông minh này được triển khai.

Các em được cung cấp sườn trên file `CrowdSaleToken_MSSV.vy`. Nhiệm vụ của các em hoàn thành theo mô tả ở phía trên.